Green Logistics Management System

Part 3 - Technical Reflection Report

Moving the system to a Service-Oriented Architecture, containerising it, and adding automated tests

Module: PROG7311 - Programming 3A (Enterprise Application Development)
Assessment: Portfolio of Evidence - Part 3
Date: 31 May 2026

1. Introduction

For Part 2 I built the Green Logistics Management System as a single ASP.NET Core MVC application where the controllers talked straight to the database. It worked, but everything lived in one project, which is fine for a prototype and not so fine once you start thinking about how a real company would run and scale it. Part 3 was about fixing that. I split the system into two separate applications - a Web API that does all the data work, and an MVC client that just handles the screens and calls the API over HTTP. On top of that I added JWT login, wrote automated tests against the API, and packaged the whole thing (database, API and web app) into Docker containers that start together with one command.

This report walks through what I changed and why, and then reflects on the two questions the brief asked me to think about: why automated testing matters so much in a CI/CD pipeline, and how Docker keeps an application behaving the same way across development, testing and production.

2. Moving from one big project to a Service-Oriented Architecture

The biggest change in Part 3 is that the presentation layer and the service layer are now two different things instead of one. What used to be a single project is now three:

- GreenLogisticsMS.Api - the back end. This is the only project that has an Entity Framework Core DbContext and the only one that ever touches SQL Server. It exposes REST endpoints (GET, POST, PATCH, DELETE) and hands back JSON.
- GreenLogisticsMS.Web - the front end. I removed every database package from it. Its controllers now go through a typed HttpClient I called ApiClient, which sends the request to the API and attaches the user's JWT token.
- GreenLogisticsMS.Tests - the unit and integration tests that check the API actually does what it's supposed to.

Why bother splitting it up? The honest answer is that it makes the system far more flexible. Because the API stands on its own, I could later point a mobile app or another company's system at the exact same endpoints without touching the web app at all. The web side can also be scaled out without every copy opening its own database connection. The part I like most, though, is that the business rules now live in one place. A rule like "you can't raise a service request against an expired contract" sits inside the API, so it can't be accidentally skipped or duplicated differently in another client - the API always has the final say.

2.1 Fixing the things I lost marks on in Part 2

The Part 2 feedback pointed out a few specific problems, so I made sure Part 3 dealt with each of them rather than ignoring them:

- "No migration script provided." I added a proper EF Core migration to the API, and it runs automatically when the API starts up (with a retry loop so it waits for the database container to be ready).
- "No logging in to the system." There's now JWT authentication. The API returns 401 if you call it without a valid token, and the MVC app has a real login page that stores the token in session.
- "Controllers call DbContext directly / tightly coupled." The MVC controllers don't even reference EF Core anymore - they only know about IApiClient, so the coupling is gone.
- "Workflow logic has bugs." The contract status workflow now goes through PATCH /api/contracts/{id}/status with the rules enforced server-side, and I wrote tests around it so I can prove it behaves.

3. Why automated testing matters in a CI/CD pipeline

A CI/CD pipeline builds, tests and (when you're ready) deploys the app automatically every time code gets pushed. The whole point is speed - but speed without a safety net is how things break in production. Automated testing is that safety net, and working on this part made that click for me.

3.1 It catches things I'd never spot by hand

If I changed the currency-conversion code, I might not realise I'd accidentally broken contract filtering somewhere else. In a pipeline, the full test suite runs on every commit, so that kind of mistake fails the build straight away instead of reaching a user weeks later. Bugs also get more expensive the longer they hide, so catching them at commit time saves a lot of pain.

3.2 Integration tests check the parts unit tests can't see

My unit tests cover the individual classes - the currency maths and the Factory, Observer and Strategy patterns. The integration tests go a step further: they actually boot the API in memory using WebApplicationFactory and hit the real endpoints. For example, they check that GET /api/contracts comes back as 200 OK with JSON that isn't null, that calling it without logging in gives a 401, and that trying to add a service request to an expired contract returns a 400. Those

tests prove that routing, authentication, EF Core and my business rules are all wired together correctly, which is exactly the stuff a unit test on its own would miss.

3.3 Confidence, and tests that double as documentation

- Confidence: a green test run is solid proof the system still works, which is what makes it safe to release often.
- Documentation: a test called PostServiceRequest_OnExpiredContract_IsRejected explains a business rule more clearly than a comment ever would, and it can't go out of date without failing.
- Repeatability: my integration tests use an in-memory database, so they run the same on my laptop as they would on a CI server, with nothing extra to install.

Basically, testing turns "I think it works" into "the pipeline proves it works," and that's the difference that lets a team ship quickly without constantly breaking things.

4. Containerisation, and the end of "but it works on my machine"

Anyone who's shared code knows the classic problem: it runs perfectly for you and then falls over on someone else's setup because they've got a different .NET version, a missing dependency or a different OS. Docker gets rid of that by shipping the app together with everything it needs to run.

4.1 How Docker keeps things consistent

- Each service has a Dockerfile that pins exact base images (mcr.microsoft.com/dotnet/aspnet:8.0 and mssql/server:2022). The same image moves unchanged from dev to test to production, so the runtime is identical everywhere.
- I used a multi-stage build, so the app compiles in the big SDK image but only the published output gets copied into a slim runtime image. That keeps the final image small and reproducible.
- docker-compose.yml describes the whole setup - sql-server-db, glms-backend-api and glms-frontend-web - plus the internal network they use to find each other. One "docker compose up" rebuilds the entire stack on any machine that has Docker.

4.2 Getting the containers to talk and start in the right order

Inside the Compose network the containers reach each other by name instead of localhost - the API's connection string points at Server=sql-server-db and the web app's ApiBaseUrl is http://glms-backend-api:8080/. SQL Server takes a little while to be ready, so the database container has a health check, the API waits for that health check, and the API also retries its migration a few times. That combination is what stopped the containers from racing each other on startup, which was something I had to work through to get a clean launch.

4.3 The same build everywhere

The end result is that what I run locally, what the pipeline would test, and what gets deployed are the exact same image. "It works on my machine" turns into "it works in the container," and the container behaves the same wherever it runs - which is really the whole idea behind a cloud-native deployment.

5. Conclusion

Part 3 took my Part 2 prototype and turned it into something that looks a lot closer to a real, modern system. Splitting it into an API and an MVC client gave it a clean separation between the service and presentation layers; JWT login closed the "no login" gap from Part 2; the EF Core migration makes the database reproducible; the automated tests guard the API's behaviour the way a CI/CD pipeline would need; and Docker Compose ties the database, API and web app into one stack that starts with a single command. Working through it also gave me a much better feel for why teams lean on automated testing and containerisation - they're what make fast, reliable releases possible instead of risky.

6. Reference List

Docker Inc. 2024. Compose file reference. [Online]. Available at: https://docs.docker.com/compose/compose-file/ [Accessed 31 May 2026].

Docker Inc. 2024. Multi-stage builds. [Online]. Available at: https://docs.docker.com/build/building/multi-stage/ [Accessed 31 May 2026].

Freeman, A. 2022. Pro ASP.NET Core 6: Develop Cloud-Ready Web Applications Using MVC, Blazor, and Razor Pages. 9th ed. New York: Apress.

Gamma, E., Helm, R., Johnson, R. & Vlissides, J. 1995. Design Patterns: Elements of Reusable Object-Oriented Software. Boston: Addison-Wesley.

Humble, J. & Farley, D. 2010. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston: Addison-Wesley.

Microsoft. 2024. Containerize a .NET app with Docker. [Online]. Available at: https://learn.microsoft.com/en-us/dotnet/core/docker/build-container [Accessed 31 May 2026].

Microsoft. 2024. Create web APIs with ASP.NET Core. [Online]. Available at: https://learn.microsoft.com/en-us/aspnet/core/web-api/ [Accessed 31 May 2026].

Microsoft. 2024. Configure JWT bearer authentication in ASP.NET Core. [Online]. Available at: https://learn.microsoft.com/en-us/aspnet/core/security/authentication/jwt [Accessed 31 May 2026].

Microsoft. 2024. Integration tests in ASP.NET Core. [Online]. Available at: https://learn.microsoft.com/en-us/aspnet/core/test/integration-tests [Accessed 31 May 2026].

Microsoft. 2024. Migrations Overview (EF Core). [Online]. Available at: https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/ [Accessed 31 May 2026].